LAPORAN PRAKTIKUM

IOT II

Disusun oleh:

Nama : Salim Suheri Sitompul

NPM : 25782053

Kelas : TRI 3B

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

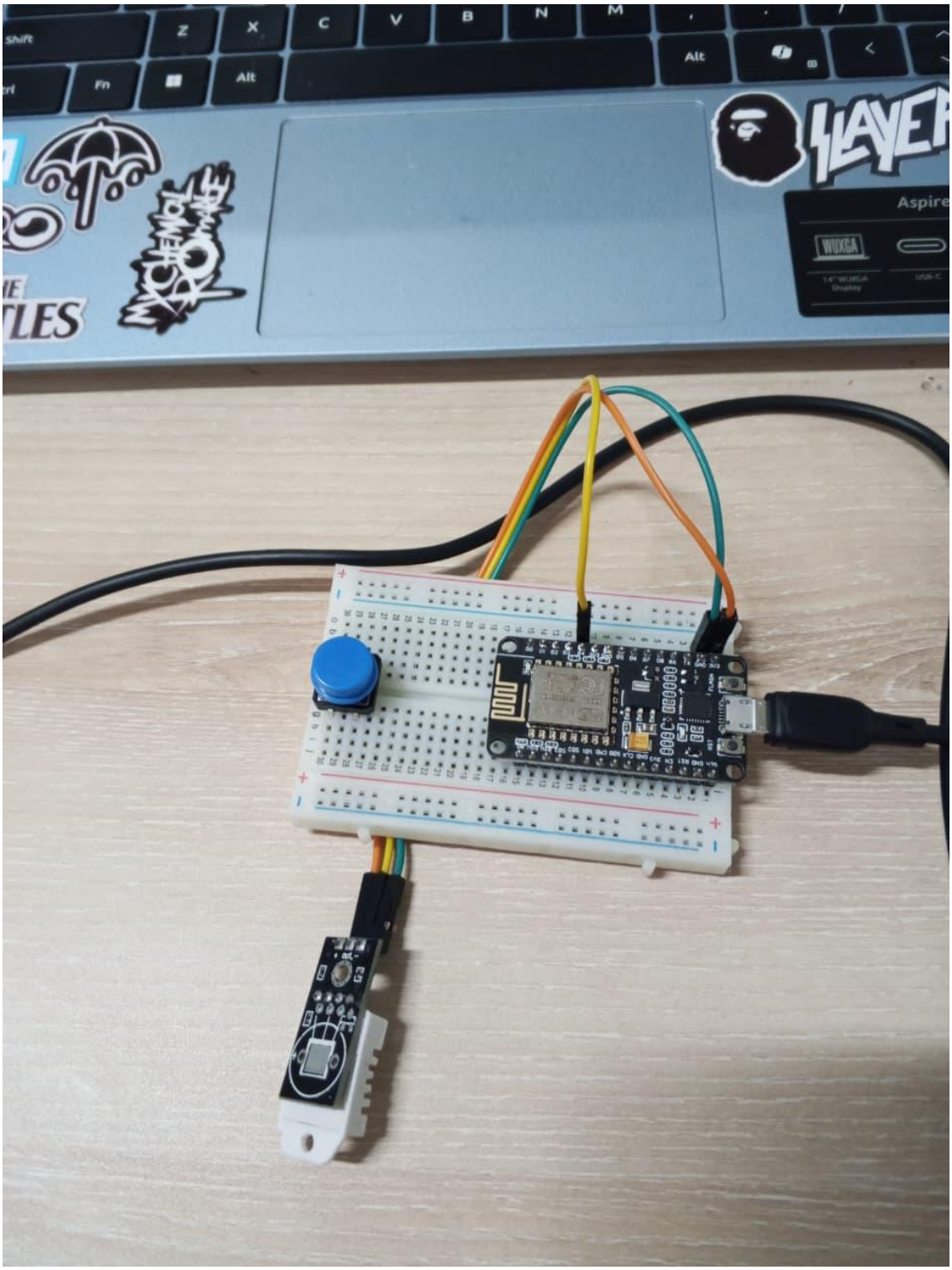
JURUSAN TEKNOLOGI INFORMASI

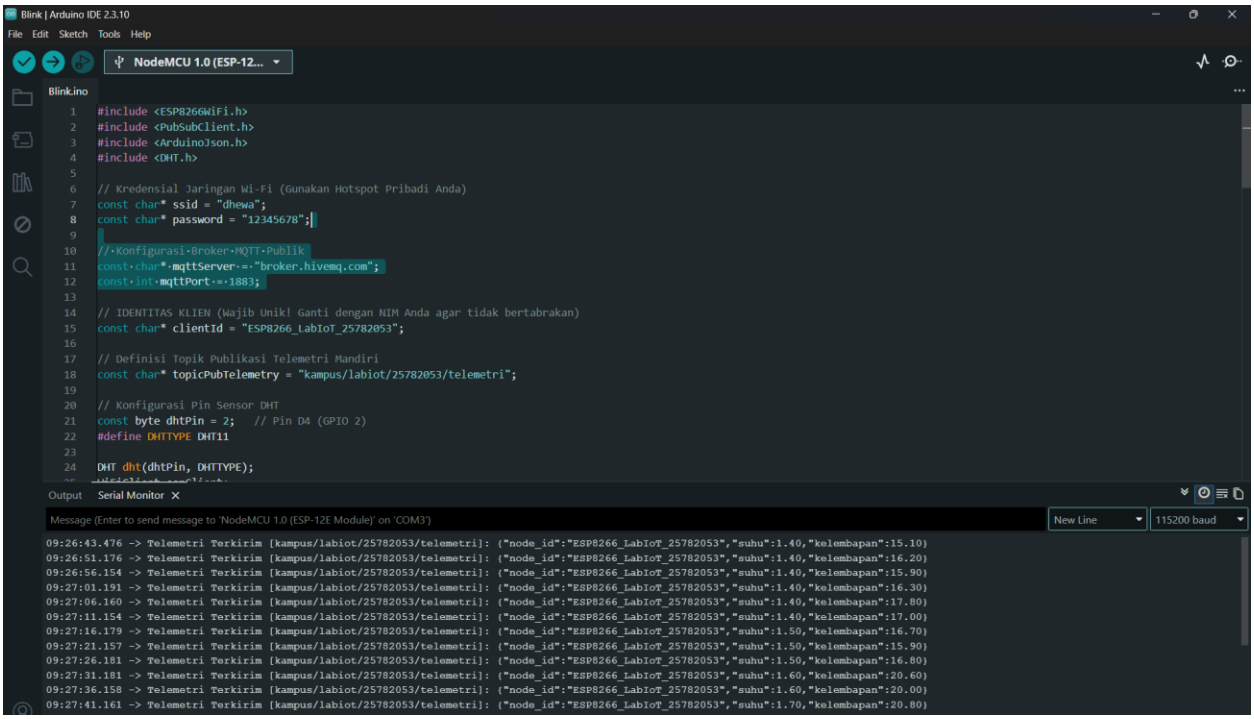
POLITEKNIK NEGERI LAMPUNG

2026

Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON





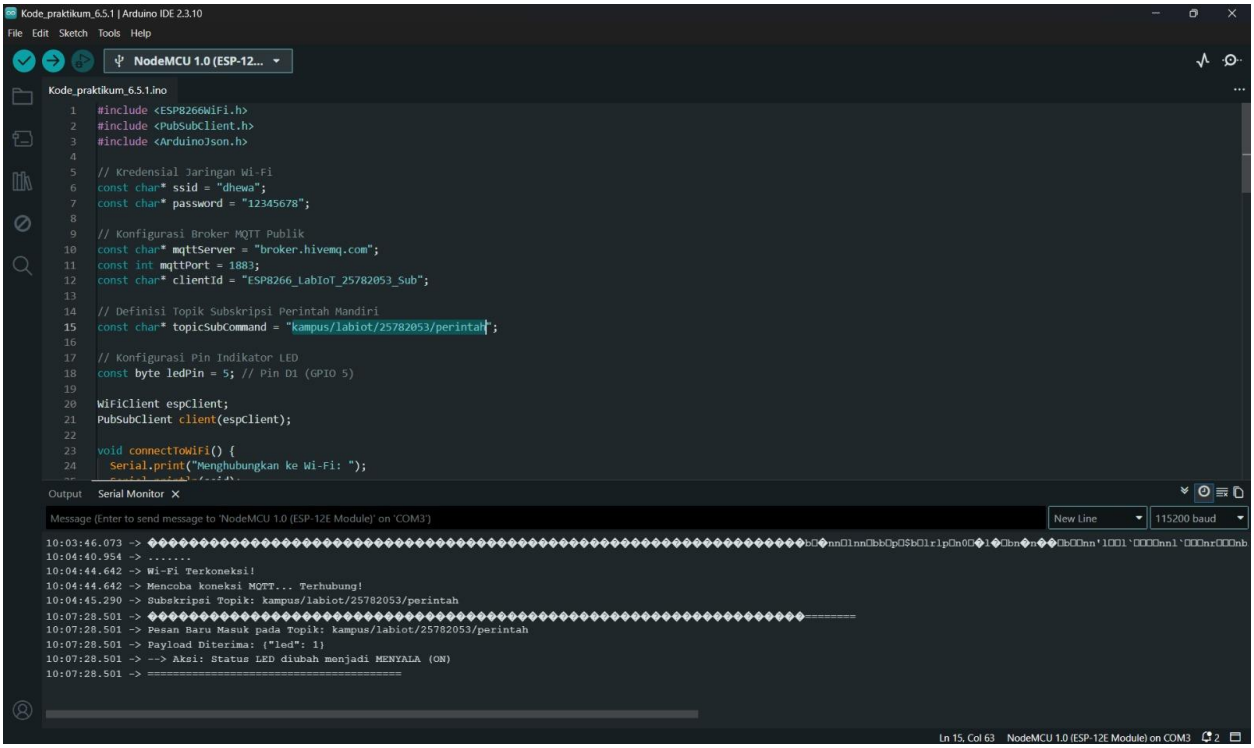
Langkah Kerja Rangkaian:

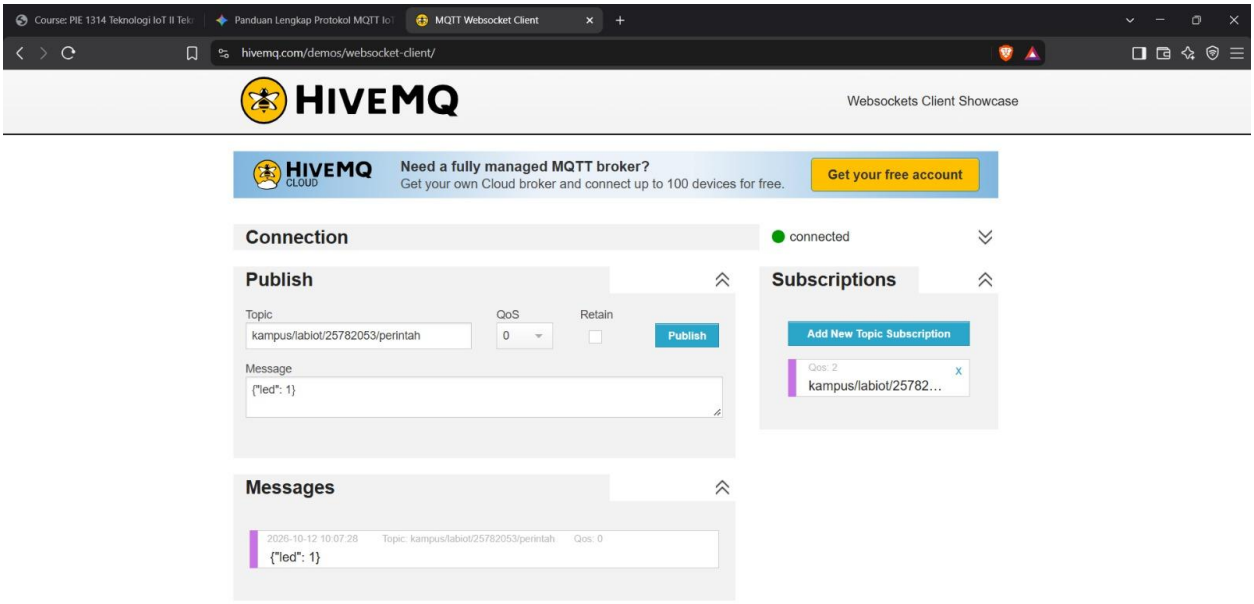
1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. Sensor DHT11/DHT22: Hubungkan pin VCC ke 3V3, pin GND ke GND, dan pin Data ke D4 (GPIO 2).

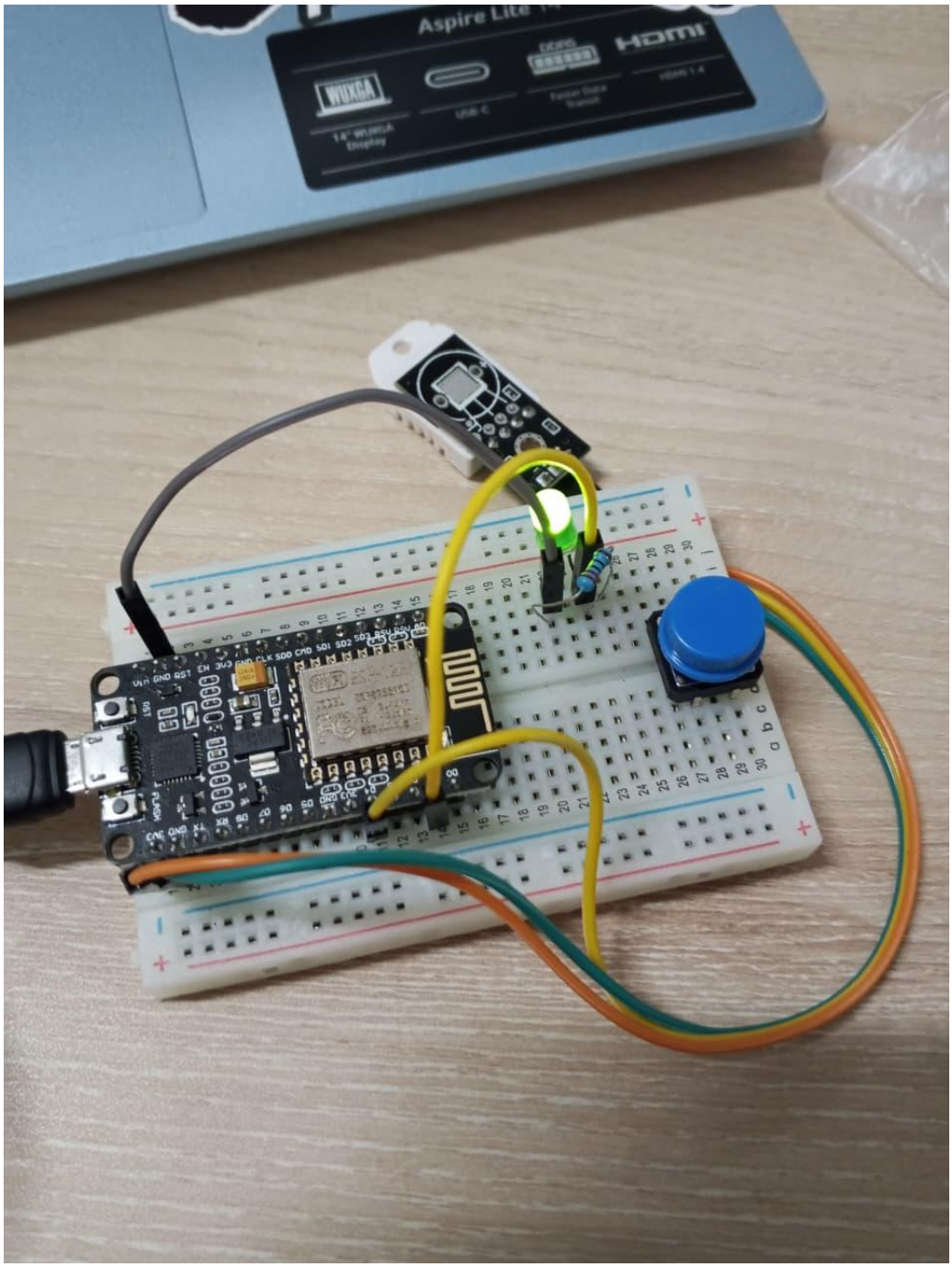
Penjelasan Singkat Kode:

- WiFiClient espClient; : Membuka antarmuka soket jaringan TCP level dasar pada cip ESP8266.
- PubSubClient client(espClient); : Menginisialisasi objek protokol MQTT yang beroperasi di atas soket jaringan espClient.
- client.setServer(mqttServer, mqttPort); : Menetapkan alamat host broker dan port layanan (1883 merupakan port standar MQTT teks terbuka / tanpa TLS).
- client.loop(); : Fungsi wajib. Rutinitas ini bertugas memproses paket TCP masuk, menjaga interval *keep-alive ping* dengan server agar koneksi tidak diputus paksa (*timeout*), serta mengelola antrean pesan.
- StaticJsonDocument<256> doc; : Mengalokasikan blok memori tetap di SRAM (sebesar 256 byte) untuk menampung pohon objek JSON tanpa memicu fragmentasi memori *heap*.
- serializeJson(doc, jsonBuffer); : Mengubah pohon objek JSON menjadi representasi string karakter padat yang siap ditransmisikan.

6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron







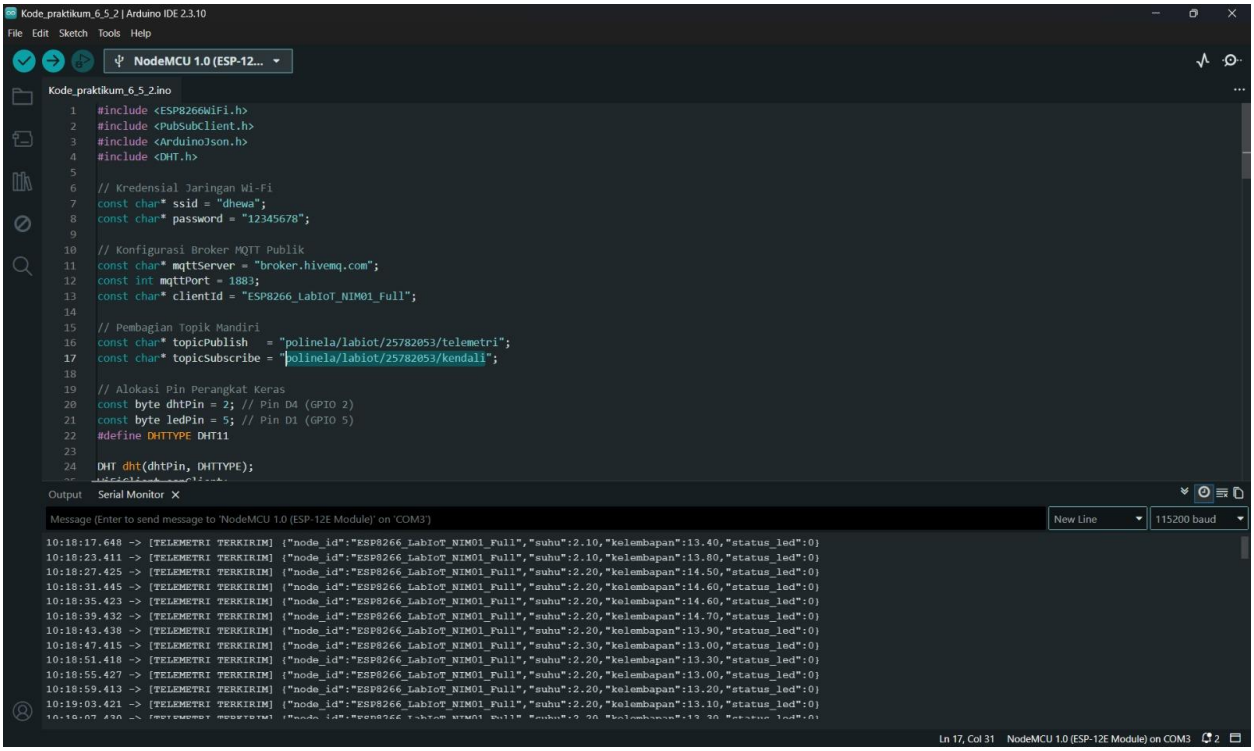
Langkah Kerja Rangkaian:

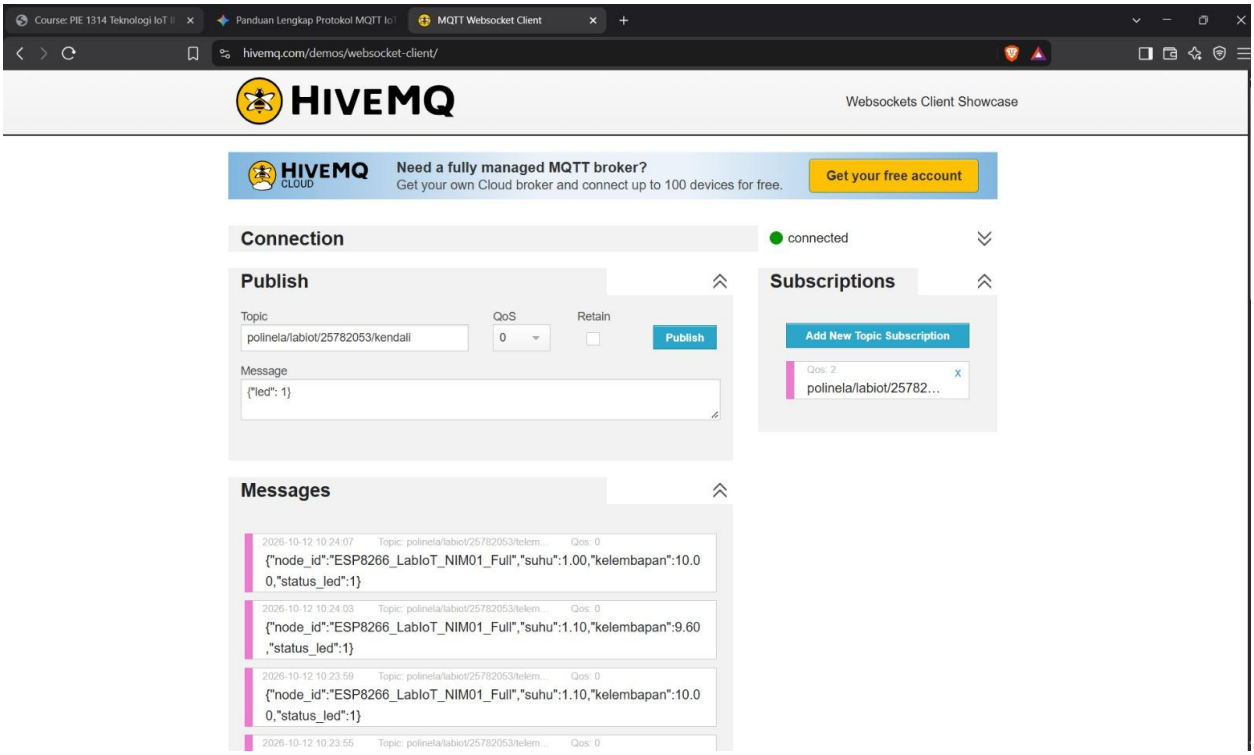
1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. LED Indikator: Hubungkan kaki Anoda (kaki panjang) LED ke pin D1 (GPIO 5), dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ωyang menuju ke GND.

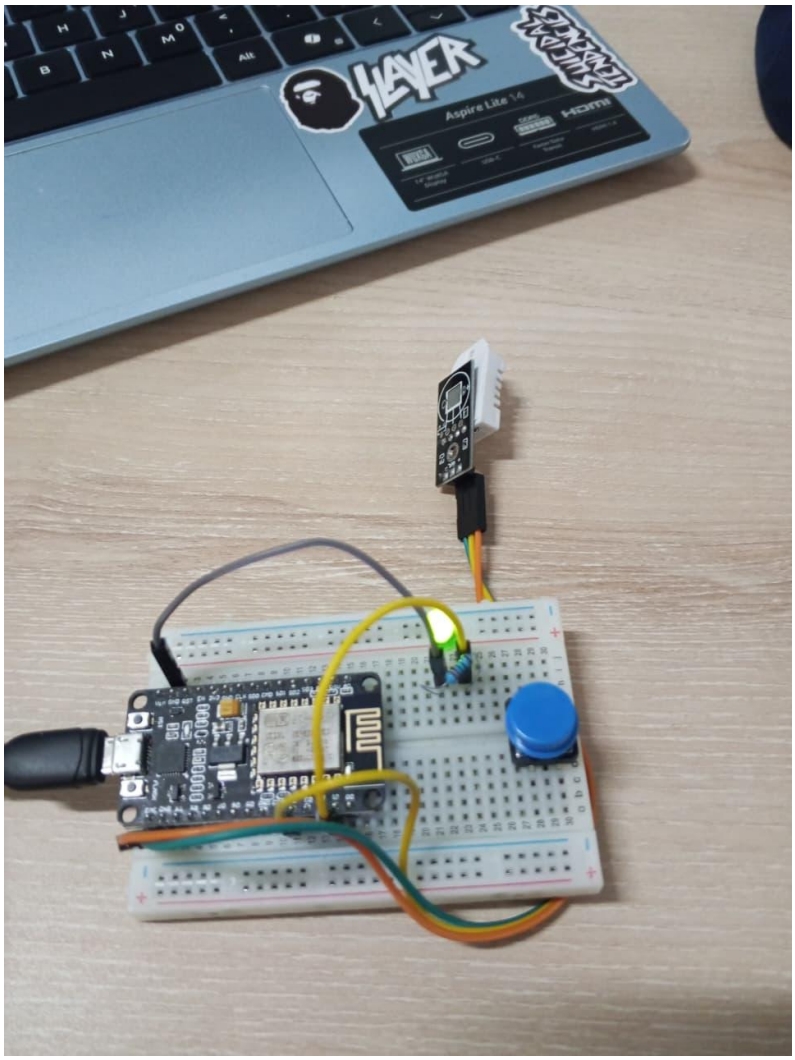
Penjelasan Singkat Kode:

- client.setCallback(onMessageReceived); : Mendaftarkan fungsi penangan paket data masuk (*callback function*). Kapan pun broker mengirimkan pesan yang sesuai dengan topik subskripsi, fungsi onMessageReceived akan dipicu secara otomatis.
- client.subscribe(topicSubCommand); : Mengirimkan instruksi subskripsi ke broker bahwa klien bersedia menerima semua pesan yang dialirkan ke topik tersebut.
- deserializeJson(doc, payloadStr); : Mengurai teks string JSON mentah dan menyusunnya kembali menjadi struktur pohon data yang kuncinya dapat diperiksa melalui doc.containsKey() dan nilainya dapat diekstrak secara langsung.

6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)




Course: PIE 1314 Teknologi IoT
Panduan Lengkap Protokol MQTT
MQTT Websocket Client
hivemq.com/demos/websocket-client/
HIVEMQ
Websockets Client Showcase
HIVEMQ CLOUD
Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.
Get your free account
Connection
connected
Publish
Topic
polinela/labiot/25782053/kendali
QoS
0
Retain
Publish
Message
{"led": 1}
Subscriptions
Add New Topic Subscription
Qos: 2
polinela/labiot/25782...
Messages
2026-10-12 10:24:07 Topic: polinela/labiot/25782053/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_NIM01_Full","suhu":1.00,"kelembapan":10.00,"status_led":1}
2026-10-12 10:24:03 Topic: polinela/labiot/25782053/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_NIM01_Full","suhu":1.10,"kelembapan":9.60,"status_led":1}
2026-10-12 10:23:59 Topic: polinela/labiot/25782053/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_NIM01_Full","suhu":1.10,"kelembapan":10.00,"status_led":1}
2026-10-12 10:23:55 Topic: polinela/labiot/25782053/telem... Qos: 0


SLAYER
Aspire Lite 14

1. Jalankan Aplikasi MQTT Explorer pada laptop Anda.
2. Konfigurasi Koneksi Broker:
    - Protocol: mqtt://
    - Host: broker.hivemq.com
    - Port: 1883
    - Klik tombol Connect.
3. Memantau Data Masuk (*Telemetry Monitoring*):
    - Amati pohon hierarki topik (*Topic Tree*) pada panel sebelah kiri.
    - Temukan dan klik cabang topik Anda: kampus →labiot →nim_anda →telemetri.
    - Anda akan melihat payload JSON diperbarui secara *real-time* setiap 4 detik. Klik pada nilai numerik untuk menampilkan grafik fluktuasi suhu dan kelembapan secara langsung.
4. Menerbitkan Instruksi Kendali (*Publish Command*):
    - Arahkan ke panel Publish di sisi kanan bawah antarmuka.
    - Masukkan topik tujuan: kampus/labiot/nim_anda/kendali
    - Pilih format muatan: JSON.
    - Masukkan muatan payload berikut:
    - {"led": 1}
    - Klik tombol Publish. Perhatikan bahwa LED indikator menyala seketika tanpa menghentikan siklus pengiriman data telemetri.
    - Kirimkan perintah pematian dengan payload {"led": 0} dan pastikan LED padam.

6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler:

   Jawaban: MQTT berbasis biner dengan ukuran *header* sangat kecil (minimal 2 bytes) tanpa pengulangan *header* teks yang besar seperti HTTP, sehingga sangat hemat kuota data seluler.

ii. Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah.

Jawaban: MQTT menggunakan koneksi TCP persisten (tetap terbuka) dan parsing biner yang ringan, sehingga mikrokontroler tidak terbeban oleh handshake TCP berulang dan parsing string kompleks seperti pada HTTP.

iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution).

Jawaban: Menggunakan arsitektur Publish-Subscribe melalui Broker. Cukup satu kali publish oleh pengirim, MQTT Broker secara otomatis meneruskan pesan secara simultan ke banyak subscriber sekaligus tanpa membebani perangkat pengirim.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.

Jawaban:

- Ekspresi Topik Subskripsi: pabrik/+/+/suhu
- Analisis & Penjelasan:
  - Level pertama (pabrik) diset secara spesifik karena data berasal dari fasilitas pabrik tersebut.
  - Level kedua (+) menggunakan single-level wildcard untuk menggantikan seluruh lini produksi (line1, line2, line3).
  - Level ketiga (+) menggunakan single-level wildcard untuk menggantikan seluruh mesin di setiap lini (m1, m2, m3, m4).
  - Level keempat diset secara spesifik ke suhu karena kita hanya ingin menyaring data metrik suhu saja, mengabaikan metrik getaran dan status.

ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.

Jawaban:

- **Ekspresi Topik Subskripsi:** pabrik/line2/m1/+
- **Analisis & Penjelasan:**
  - Level pertama (pabrik) dan level kedua (line2) dikunci secara spesifik untuk membatasi pemantauan hanya pada lini produksi kedua.
  - Level ketiga (m1) dikunci secara spesifik untuk memilih hanya mesin nomor 1 saja, sehingga mesin m2, m3, dan m4 terfilter keluar.
  - Level keempat (+) menggunakan *single-level wildcard* untuk menangkap seluruh jenis sensor atau metrik yang dikirimkan oleh mesin tersebut secara bersamaan (mencakup suhu, getaran, dan status).

iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

Jawaban:

- **Ekspresi Topik Subskripsi:** pabrik/#
- **Analisis & Penjelasan:**
  - Level pertama (pabrik) bertindak sebagai *root* atau akar topik utama.
  - Karakter # (*multi-level wildcard*) diletakkan setelah tanda garis miring (/) di belakang level pabrik.
  - Penggunaan tanda # ini secara otomatis memerintahkan broker MQTT untuk meneruskan **semua** pesan apa pun yang masuk ke dalam hierarki di bawah kata pabrik, tanpa peduli apa pun lini produksinya (line1-3), apa pun jenis mesinnya (m1-4), dan apa pun jenis sensornya (suhu, getaran, status). Topik ini biasanya digunakan untuk kebutuhan pusat pemantauan utama (*master dashboard* atau sistem *logging* pusat).

Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).

Jawaban:

```
// 1. Hitung lama waktu aktif sistem dalam satuan detik
unsigned long uptime_detik = millis() / 1000;

// 2. Tambahkan parameter "uptime_detik" ke dalam payload JSON
String payload = "{";
payload += "\"temperature\": " + String(temperature) + ",";
payload += "\"humidity\": " + String(humidity) + ",";
payload += "\"uptime_detik\": " + String(uptime_detik);
payload += "}";
```

Penjelasan:

1. **Fungsi millis()**:
   - Ini adalah fungsi bawaan di Arduino untuk menghitung berapa lama mikrokontroler menyala (berjalan) sejak pertama kali di-hidupkan atau di-*reset*, dalam satuan **milidetik**.
2. **Dibagi 1000 (/ 1000)**:
   - Karena 1 detik sama dengan 1.000 milidetik, maka nilai dari millis() dibagi 1000 agar waktu aktif sistem berubah dari milidetik menjadi **detik** (uptime_detik).
3. **Penyusunan ke JSON**:

- Nilai angka tersebut kemudian diubah menjadi format teks (String(uptime_detik)) dan diselipkan ke dalam string payload JSON agar ikut terkirim bersama data sensor ke MQTT broker.

  Dengan cara ini, sistem pemantauan bisa tahu sudah berapa lama mikrokontroler tersebut menyala sejak terakhir kali aktif.

ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

Jawaban:

```
// 1. Ambil alamat IP lokal mikrokontroler dalam format string
String ip_lokal = WiFi.localIP().toString();

// 2. Tambahkan parameter "ip_lokal" ke dalam payload JSON
String payload = "{";
payload += "\"temperature\": " + String(temperature) + ",";
payload += "\"humidity\": " + String(humidity) + ",";
payload += "\"uptime_detik\": " + String(uptime_detik) + ",";
payload += "\"ip_lokal\": \"" + ip_lokal + "\"";
payload += "}";
```

Penjelasan:

1. WiFi.localIP().toString():
   - Perintah ini digunakan untuk mengambil alamat IP lokal yang diberikan oleh router Wi-Fi kepada mikrokontroler (misalnya 192.168.1.15), lalu mengubahnya dari tipe data objek IP menjadi format teks biasa (*String*).
2. Format Teks pada JSON (\"):
   - Karena tipe data IP berupa teks/string, penulisan di dalam format JSON wajib diapit oleh tanda kutip ganda ("). Di dalam kode C++, tanda kutip ganda ditulis menggunakan *escape character* yaitu \" agar tidak dianggap menutup baris kode program secara keliru.

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

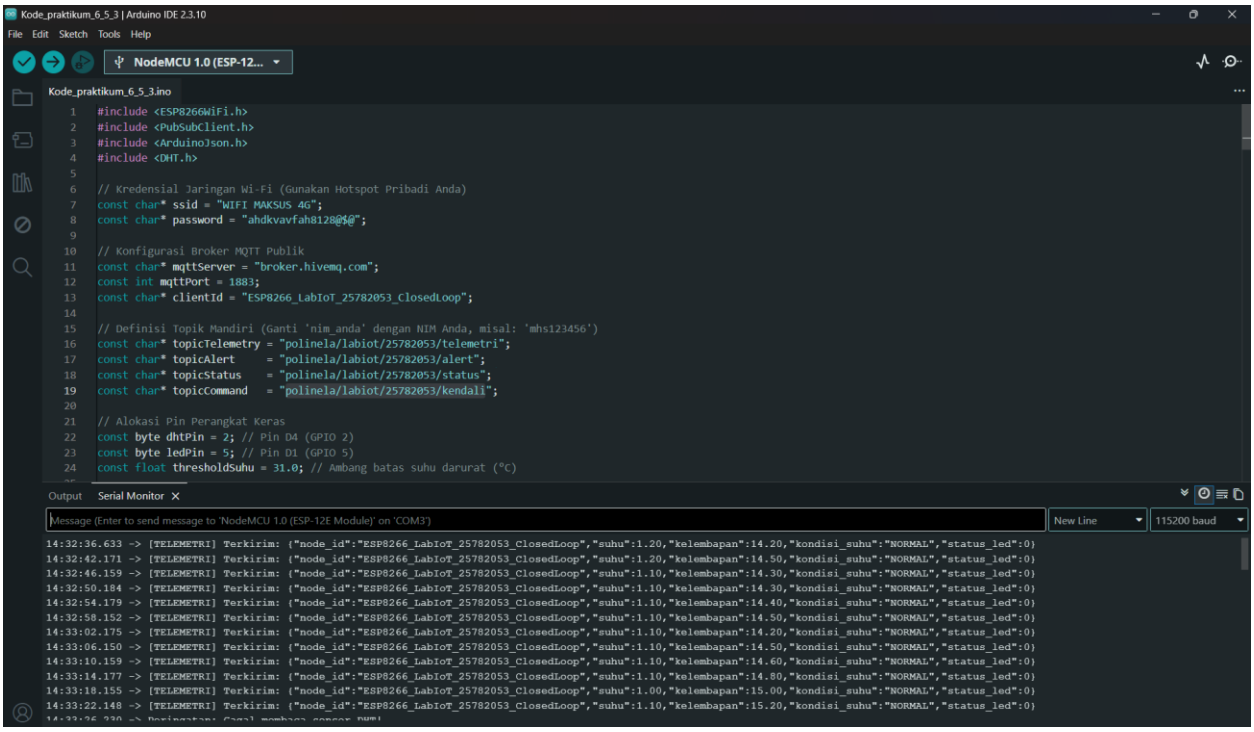
i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

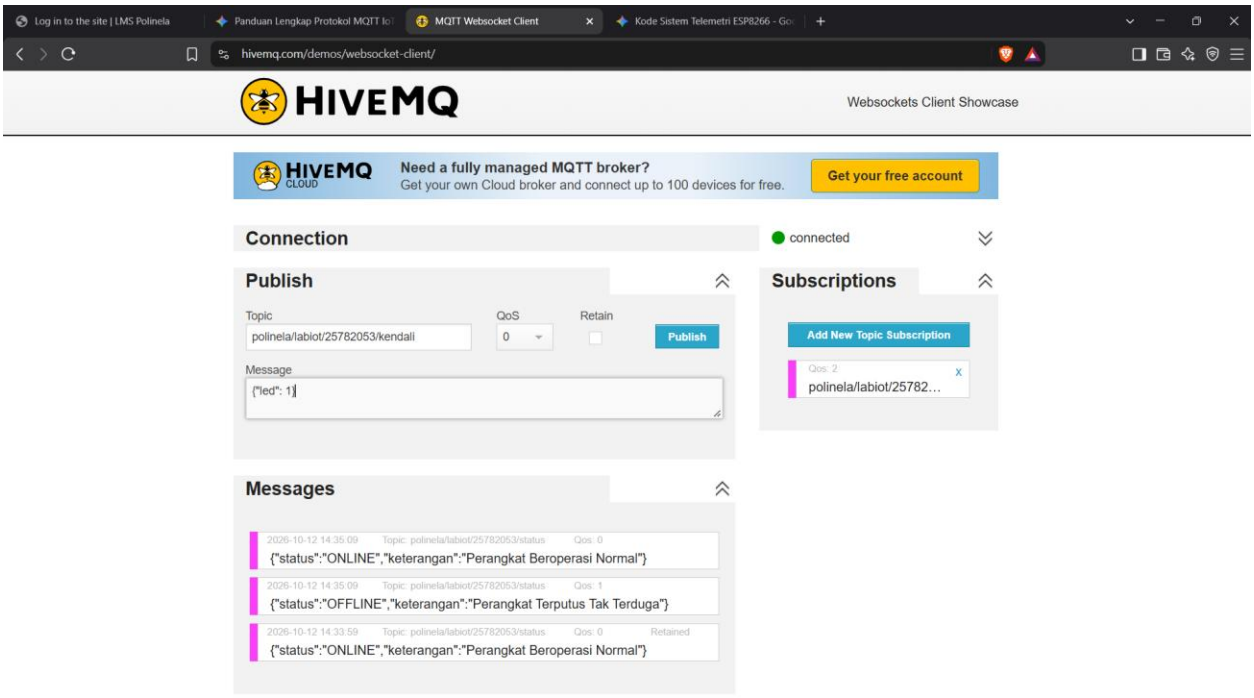
Jawaban: Penyisipan delay(6000) membuat mikrokontroler terhenti total selama 6 detik tanpa menjalankan fungsi client.loop(). Akibatnya, responsivitas terhadap perintah callback menjadi tertunda, dan perangkat berisiko mengalami disconnect dari broker karena gagal mengirimkan sinyal keep-alive ping tepat waktu.

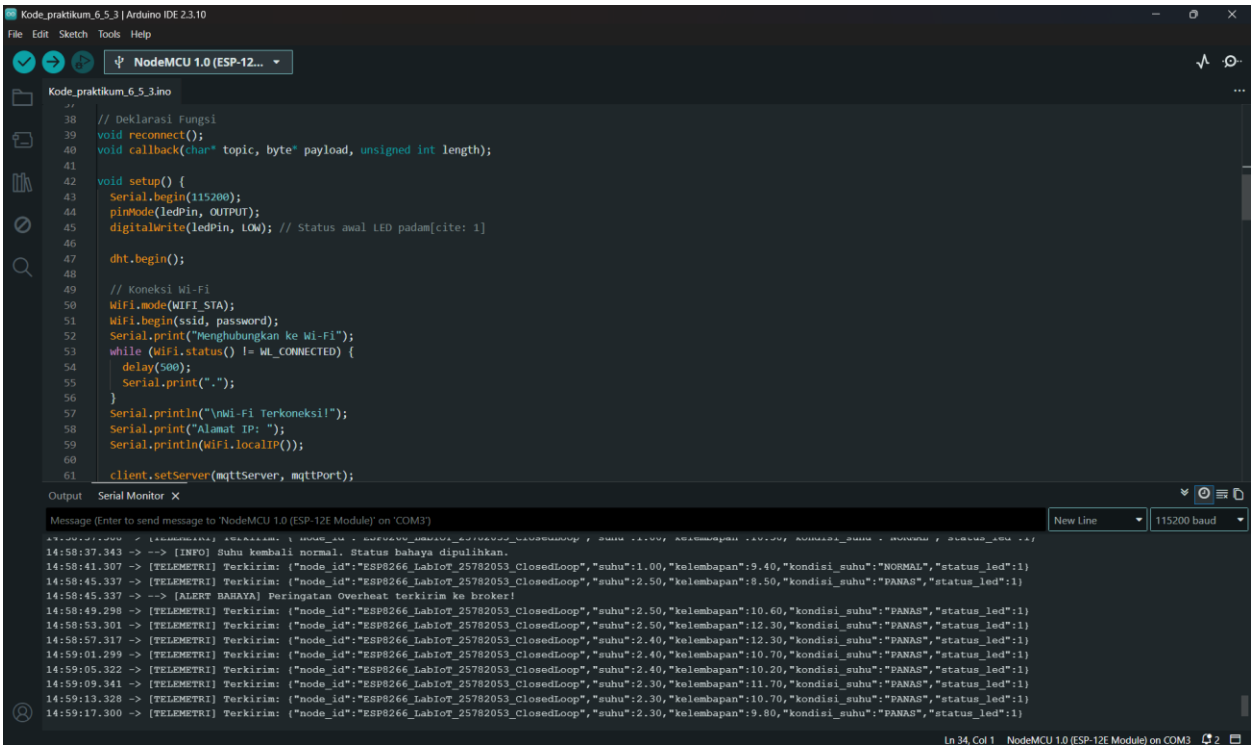
ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Jawaban: Fungsi reconnect() bekerja dengan memeriksa status koneksi menggunakan client.connected(). Saat terdeteksi terputus, fungsi ini akan melakukan perulangan untuk mencoba menghubungkan kembali perangkat ke MQTT broker melalui client.connect(). Setelah berhasil terhubung kembali, fungsi ini langsung menjalankan client.subscribe() untuk mendaftarkan ulang topik langganan agar perangkat dapat kembali menerima data dan perintah dari sistem secara normal tanpa perlu merestart perangkat.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri







Tujuan: Mampu memadukan akuisisi sensor suhu dan kelembapan, serialisasi JSON dinamis berdasar kondisi lingkungan, subskripsi kendali indikator LED jarak jauh, kanal peringatan darurat independen (*dedicated alert topic*), dan pendaftaran pesan wasiat *Last Will and Testament* (LWT) secara mandiri.

Instruksi Perangkaian:

1. Pertahankan konfigurasi perangkat keras dari Praktikum 3:
    - Sensor DHT11/DHT22 terhubung ke pin **D4 (GPIO 2)** (VCC ke 3V3, GND ke GND).
    - LED Indikator + Resistor 220 Ωterhubung ke pin **D1 (GPIO 5)** (Katoda resistor ke GND).

Instruksi Pemrograman (Rule Engine & Safety Logic):

1. **Klasifikasi Status Lingkungan pada Telemetri Rutin:**
   Modifikasi dokumen JSON telemetri rutin (kampus/labiot/nim_anda/telemetri) agar menambahkan kunci "kondisi_suhu" bertipe string:
    - JIKA suhu $\leq 31.0°C$, isi nilai menjadi "NORMAL".
    - JIKA suhu $> 31.0°C$, isi nilai menjadi "PANAS".
2. **Kanal Peringatan Bahaya Mandiri (*Dedicated Alert Topic*):**
   Implementasikan logika pengawasan ambang batas kritis lokal (*edge safety engine*):
    - **JIKA** suhu terukur melebihi ambang batas $31.0°C$:
        - Sistem harus segera memublikasikan paket peringatan bahaya ke topik khusus: kampus/labiot/nim_anda/alert dengan format JSON:
        - {
        - "status": "BAHAYA",
        - "parameter": "OVERHEAT",
        - "nilai_suhu": 31.8
        - }
        - Secara otomatis menyalakan LED indikator peringatan bahaya tanpa menunggu perintah dari operator luar.
        - Berikan proteksi penanda (*flag / state latch*) agar pesan peringatan ini tidak membanjiri broker setiap milidetik, melainkan dikirimkan teratur saat kondisi bahaya terdeteksi pertama kali.
3. **Mekanisme Last Will and Testament (LWT):**
   Konfigurasikan fungsi client.connect() dengan menyertakan parameter pendaftaran pesan wasiat (*Last Will*):
    - **Topik LWT:** kampus/labiot/nim_anda/status
    - **Pesan Wasiat LWT:**
    - {

- "status": "OFFLINE",
- "keterangan": "Perangkat Terputus Tak Terduga"
- }
- **QoS LWT:** 1
- **Retain Flag:** true
- Sesaat setelah mikrokontroler berhasil terhubung ke Broker, publikasikan pesan status aktif (*online message*) ke topik status yang sama:
- {
- "status": "ONLINE",
- "keterangan": "Perangkat Beroperasi Normal"
- }
- Uji mekanisme ini: Buka MQTT Explorer, amati topik status (terbaca ONLINE), lalu cabut kabel data USB NodeMCU secara tiba-tiba. Amati broker: pesan wasiat OFFLINE akan muncul secara otomatis beberapa saat kemudian.

4. **Kendali Manual Override Saat Normal:**
   Saat suhu berada di bawah ambang batas bahaya, operator luar tetap dapat menyalakan atau mematikan LED secara manual melalui pengiriman payload {"led": 1} atau {"led": 0} ke topik kampus/labiot/nim_anda/kendali.